

DATA 442/610: Deep Learning and Neural Networks

Assignment 1: Building a NN from Scratch

Assigned: Sep 9, 2026; Due: Oct 2, 2026

Instructions

This assignment will be graded out of 100 points. It contains three problems. You are welcome to solve as many problems as you wish. Your score will be calculated based on the problems you turn in, up to a maximum of 100 points. **Any bonus questions will be scored and added on top of the 100-point grade total.**

- **Example:** You try Problem 1 and Problem 2, scoring 40 points on each. You do not do Problem 3. Your grade will be 80/100.
- **Example:** You try Problem 1 and Problem 2, scoring 40 points on each. You do part of Problem 3 and score 10 points. Your grade will be 90/100
- **Example:** You try all three problems, scoring 45 points on Problem 1, 35 points on Problem 2, and 30 points on Problem 3. Your grade will be 100/100

Please submit each problem you attempt as a separate Jupyter notebook with labeled headings/subheadings for each question. This assignment will be graded based on demonstrated understanding of the core concepts. You do not need to achieve the best possible neural network accuracy. High marks will come from thoughtful documentation and explanation of *what* you are attempting, *how* you expect it to perform, and *why* you believe it will behave like it does. **Please comment your code.**

This assignment requires working out some intermediate mathematical steps. You are encouraged to show your work on any derivations underlying your code, and include this alongside your submission. You may submit any mathematical derivations as typset equations in your Jupyter Notebooks. If you prefer not to type out that many equations, you may include a scanned PDF of handwritten calculations.

Special Note for Assignment 1

The title of this assignment is "Building a NN from Scratch." Deep learning libraries such as PyTorch, TensorFlow, Keras, Theano, and Jax are not permitted. **Scikit-Learn (import sklearn) is allowed, with the exception of sklearn.neural_network which is not permitted.** Importing a deep learning library in a Jupyter notebook will result in a score of 0 for that entire problem.

If you are unsure of whether a library is permitted, please ask during class or office hours, or alternatively via the discussion forum or email.

Problem 1 (50 points) - Fashion-MNIST classification from scratch

In this problem you will develop a 2-layer MLP to classify the Fashion MNIST dataset. You can download the dataset and read its documentation here: <https://www.kaggle.com/datasets/zalando-research/fashionmnist>. Alternatively, you can load it using the following code snippet:

```
from sklearn.datasets import fetch_openml
fashion_mnist = fetch_openml("Fashion-MNIST", version=1)

X = fashion_mnist.data.to_numpy(dtype=np.float32)
y = fashion_mnist.data.to_numpy(dtype=np.int8)
```

Part 1: Data Preprocessing (10 points)

Question 1 (5 points): Visualize one example from each class with the appropriate label.

Question 2 (3 points): Perform a principal component analysis (PCA) of the input data. You may use `sklearn.decomposition.PCA`. Visualize the dataset along the first two principal component dimensions. Comment on the data distribution. How many principal components are necessary to capture 80% of dataset variation?

Question 3 (2 points): Perform a train/validation/test split and scale the input features using either min-max scaling or standard scaling. You may use `sklearn.preprocessing`.

Part 2: Model training (40 points)

Implement a MLP with two hidden layers to classify the MNIST-Fashion dataset. Use the softmax activation on the output layer

$$\sigma(z_i) = \frac{\exp z_i}{\sum_j \exp z_j}$$

Train using the **categorical cross entropy loss**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_j^{(i)} \ln \hat{y}_j^{(i)}$$

Here, N is the number of examples and C is the number of classes. For the i -th example, $y_j^{(i)}$ is the one-hot encoded label for class j and $\hat{y}_j^{(i)}$ is the predicted probability for that class.

You may use the notebook from the in-class lecture as a template.

Question 1 (10 points): Modify the model initialization to include a second hidden layer of neurons and neuron weights, and modify the "forward" function to include the softmax output.

Question 2 (5 points): Derive `d_loss__d_z_output`, the gradient of the loss with respect to the final layer pre-activation.

Question 3 (15 points): Modify the "backward" function of your MLP to explicitly compute and return gradients of the loss with respect to `w_output`, `b_output`, `w_2`, `b_2`, `w_1`, `b_1`. Be sure

to comment your code to keep track of the shapes of intermediate arrays!

Question 4 (5 points): Modify the training loop to compute multi-class accuracy and adjust the weights of the two-layer MLP.

Question 5 (10 points): Initialize and train your model. Aim for a minimum training accuracy of 95%. We recommend starting with the following hyperparameters, although you may adjust them to improve performance.

- Hidden layer width: 256
- Hidden layer activation: ReLU
- Learning rate: 0.001
- Epochs: 50

Plot the training/validation loss/accuracy curves and comment on the model's performance. Be sure to check your model's test accuracy as well!

Question 6 (5 points): Re-initialize the model and train again using a different input feature scaling. If you used min-max scaling, in Part 1, now you should use standard scaling. If you used standard scaling in Part 1, use min-max scaling. How did model training and test performance change as a result? Why do you think this happened?

Problem 2 (50 points) - Hyperparameter tuning exploration

In this problem, you will investigate the effect of varying model hyperparameters on training and test performance. We will use a 2-class toy dataset for classification that you can generate using the following function. Do not normalize or standardize this dataset.

```
import numpy as np

def make_dataset(N=2048, random_seed=42):
    np.random.seed(random_seed)
    y = np.random.choice(2, size=N).astype(np.uint8)

    X = np.empty([N, 4], dtype=np.float32)
    X[:, 0] = np.random.uniform(low=-5, high=5, size=N)
    X[:, 1] = np.random.normal(loc=0., scale=2.5, size=N)
    X[:, 2] = np.random.normal(loc=0., scale=1., size=N)
    X[:, 3] = np.sqrt(X[:, 0]**2 + X[:, 1]**2 + X[:, 2]**2 + y**2)
    X[:, 3] *= np.random.choice([-1, 1], size=N)
    return X, y
```

Part 1: Data Preprocessing (4 points)

Question 1 (2 points): Visualize the dataset by plotting pairwise correlations between each input feature for each class. You may use functions such as `seaborn.pairplot`.

Question 2 (2 points): Describe the structure of the dataset. Do you think it is linearly separable?

Part 2: Polynomial Logistic Regression (6 points)

Question 1 (2 points): Implement a single-output logistic regression model fit with mini-batch stochastic gradient descent to minimize the cross-entropy loss. Include an argument `degree` that allows polynomial feature augmentation. You may use the notebook from class as a template.

Question 2 (2 points): Fit the model with `degree=1` and `degree=2`. Be sure to track loss and accuracy for both training and validation sets, and retain a held-out test set of at least 20% of the overall dataset.

Question 3 (2 points): For each model, plot the loss and accuracy curves for both training and validation sets. Also plot a confusion matrix and ROC-AUC curve on the held-out test set. Which model performs better? Why do you think this is the case?

BONUS Question (5 points): Fit additional models up to at least `degree=6` (you may go higher if you wish). Plot the test accuracy and ROC-AUC for each model as a function of the polynomial feature degree. Are there any patterns? If so, why do you think they occur?

Part 3: Single-layer NN (14 points)

Question 1 (5 points): Implement a neural network with one hidden layer. For this neural network, use the ReLU activation on the hidden layers and the `tanh` activation on the output layer

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Derive `d_h_output__d_z_output` and implement the backward pass for this neural network. As we did in class, you may use the MSE loss (even though this is a classification problem).

Question 2 (5 points): Train neural network models with hidden sizes of [16, 32, 64, 128, 256, 512, 1024]. Be sure to record training and validation loss/accuracy curves, and use the same test dataset from Part 2. Use following hyperparameters:

- batch size = 32
- epochs = 1000
- learning rate = 0.01

Question 3 (2 points): For each model, plot the loss/accuracy curves as well as a confusion matrix and ROC-AUC curve for the test dataset.

Question 4 (2 points): For each model, compute the total number of trainable parameters. Plot the test accuracy and test ROC-AUC as a function of the model size. What trends emerge? Why do you think they occur?

Part 4: Activation functions (14 points)

Question 1 (4 points): Modify your neural network by replacing `ReLU` with a “Leaky ReLU”

$$\text{Leaky ReLU}(z) = \begin{cases} z & z \geq 0 \\ \lambda z & z < 0 \end{cases}$$

Plot this activation function and compute its derivative to modify the backward pass. Continue to use `tanh` on the output layer. Train the Leaky ReLU model using a hyperparameter configuration from Part 3, and plot training/validation loss/accuracy curves, a test confusion matrix, and a test ROC-AUC curve.

Question 2 (4 points): Modify your neural network by replacing `ReLU` with a “Quadratic”

$$f(z) = z^2$$

Plot this activation function and compute its derivative to modify the backward pass. Continue to use `tanh` on the output layer. Train the Quadratic model using a hyperparameter configuration from Part 3. You may need to adjust learning rates or weight initialization to improve performance. Plot training/validation loss/accuracy curves, a test confusion matrix, and a test ROC-AUC curve.

Question 3 (4 points): Modify your neural network by replacing `ReLU` with a “Swish”

$$\text{Swish}(z) = \frac{z}{1 + e^{-z}}$$

Plot this activation function and compute its derivative to modify the backward pass. Continue to use `tanh` on the output layer. Train the Swish model using a hyperparameter configuration from Part 3. You may need to adjust learning rates or weight initialization to improve performance. Plot training/validation loss/accuracy curves, a test confusion matrix, and a test ROC-AUC curve.

Question 4 (2 points): Compare performance of models using different activation functions. Did any perform better than the others? Why do you think this might be the case?

Part 5: Regularization (12 points)

Question 1 (3 points): Using a MLP with at least 256 neurons from Part 3, apply L2 regularization. Modify the backward pass as needed.

Question 2 (3 points): Using a MLP with at least 256 neurons from Part 3, apply L1 regularization. Modify the backward pass as needed.

Question 3 (3 points): For both L1 and L2 regularization, train models with weights

$$\lambda \in [0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}]$$

Plot the training, validation, and test accuracy as a function of regularization strength.

Question 4 (3 points): Comment on how regularization affects model performance. Why do you think regularization impacts overfitting in each case?

Problem 3 (35 points) - Vanishing gradients in deep neural networks

In this problem, you will use a 2-layer MLP to investigate the phenomenon of *vanishing gradients*. This occurs when training deep neural networks with some particular problematic activation functions that cause gradient updates to shrink dramatically and slow down the learning process. Here, you will implement and analyze a MLP with Gaussian activations to get a sense for this effect. This problem will use the XOR dataset from class, which can be generated using the code snippet below.

```
def make_xor_dataset(N=128, random_seed=42):
    np.random.seed(random_seed)
    X = 2 * np.random.uniform(size=(N, 2)).astype(np.float32) - 1
    y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
    y = np.where(y, 1, 0)
    return X, y
```

Part 1: Gaussian MLP (15 points)

Implement a MLP with two hidden layers. Use Gaussian activations on the hidden layers

$$f(z) = e^{-z^2}$$

and use linear activations on the output layer. The goal will be to perform regression on the XOR dataset described in class.

You may use the notebooks from the in class lectures as a template.

Question 1 (5 points): Modify the model initialization to include a second hidden layer of neurons and neuron weights, and modify the “forward” function to include Gaussian activations on the hidden layers and linear activations on the output layer.

Question 2 (5 points): Derive $d_loss_d_z2$ and $d_loss_d_z1$ using the definition of the Gaussian activation function. Use this result to modify the “backward” function of your MLP to explicitly compute and return gradients of the loss with respect to each network parameter.

Question 3 (5 points): Train your neural network to predict the XOR dataset. Aim for a training accuracy of 95%. We recommend starting with the following hyperparameters, although you may adjust them to improve performance.

- Hidden layer width: 256
- Learning rate: 0.001
- Epochs: 5000

Part 2: Visualize the forward pass (10 points)

The Gaussian activation has a peak around $|z| = 0$ and decays sharply away from the origin. In this part, you will visualize how this may effect the behavior of input and deeper network layers.

Question 1 (6 points): Initialize the Gaussian MLP from Part 1 with 2-neuron hidden layers (`hidden_size = 2`). Generate a grid of two-dimensional inputs $\{x_1, x_2\}$ and visualize the activation functions in each layer as a function of the network inputs. You should have plots of

$h_0^{(1)}, h_1^{(1)}, h_0^{(2)}, h_1^{(2)}$ over the two-dimensional space of possible inputs.

Question 2 (2 points): Describe what happens when the inputs are very large, e.g. $x_1 > 100$. How would feature scaling affect this behavior?

Question 3 (2 points): Describe what happens when the weights are large, e.g. $w_{00}^{(1)} = 10$. How does the bias term affect the layer activations?

Part 3: Visualize the backward pass (10 points)

Question 1 (6 points): In Part 1, you derived the gradients of the loss with respect to each layer's pre-activation. Plot these gradients as a function of the network inputs. You should have plots of $\partial\mathcal{L}/\partial z_0^{(1)}, \partial\mathcal{L}/\partial z_1^{(1)}, \partial\mathcal{L}/\partial z_0^{(2)}, \partial\mathcal{L}/\partial z_1^{(2)}$ over the two-dimensional space of possible inputs.

Question 2 (2 points): Consider again the case when the weights are large. Describe the gradient patterns in this case.

Question 3 (2 points): Plot the Gaussian activation $f(z)$ and its derivative $f'(z)$. Then, compute and plot the composition $f(f(z))$ and its derivative. How does the gradient of the nested function $\partial f(f(z))/\partial z$ compare to $\partial f(z)/\partial z$? Comment on how this might contribute to the "vanishing gradient" effect in deep NNs with this activation.

BONUS Question (5 points): Consider further nested compositions such as $f^{(3)}(z) = f(f(f(z)))$, $f^{(4)}(z) = f(f(f(f(z))))$, etc. Plot these functions up to $f^{(10)}(z)$ and derive their derivatives with respect to the pre-activation z . Comment on the behavior of gradients of higher-order compositions. How might this affect the training of deep neural networks?